

雅可比与矩阵求导实验

从逐元素手算走到两层批次反向传播

先修011。实验在CPU使用NumPy，读取本地原创的无量纲合成配置，不联网、不需要GPU或自动微分库。目标是手算一个两层批次网络的全部梯度，再用显式雅可比、JVP、VJP、独立有理数oracle和中心差分交叉检查。结果只说明这些数学与实现检查，不证明模型训练效果。

先阅读正文的行向量约定：数据按行存储，J的行是输出、列是输入；标量损失对一个矩阵的梯度按原矩阵shape排列。参数展开顺序固定为W1、b1、W2、b2，各自按行展开。

A 在运行前写出纸笔基准

读取data/network.json，写出X、Y和四组参数的shape。数据共两行，每行两个输入；一行偏置被两行样本共享，不是两套独立参数。先手算A、H、P、 $E=P-Y$ 和L，再填全部参数梯度及G_X。

检查点A：A第一行为(2,1.5)，第二行为(-0.5,2.5)；预测为(0,-11.75)^T，损失29.140625。请解释为什么除数是4，为什么上游 $G_P=(P-Y)/2$ ，而不是 $(P-Y)/4$ 。

B 从空状态运行

在已有Python与NumPy的环境中进入本单元目录执行：

```
python experiment.py --output outputs
python test_experiment.py
```

脚本生成results.json、difference_scan.csv和environment.json。Notebook重新启动内核后从头运行，把同名文件写入notebook_outputs；重复执行会覆盖相应输出目录中的这三个文件，不改原始配置。若要保留旧结果，使用不同的--output目录。已提交的outputs为一组脚本参考输出，Notebook单元格保留真实执行结果。

检查点B：脚本损失为29.140625，显式雅可比shape为[2,9]；测试报告7组、100个独立Fraction夹具、31个错误输入拒绝案例，以及实际执行的文档代码块数。统计是有限的检查记录，不能替代正文对公式成立条件的说明。

C 先核对每个中间量

```
import numpy as np
from experiment import load_config, forward, gradients, loss
cfg, X, params, Y = load_config()
v = forward(X, params)
g = gradients(X, params, Y)
assert X.shape == (2, 2) and Y.shape == (2, 1)
np.testing.assert_array_equal(v['A'], [[2, 1.5], [-0.5, 2.5]])
np.testing.assert_array_equal(v['P'], [[0], [-11.75]])
assert loss(X, params, Y) == 29.140625
```

forward返回的X、参数和中间值是本次运算的副本。检查A到H用的是A*A，而不是A@A。把W1或b1改变后必须重新前向，不可沿用旧A。这种缓存混用即使shape一致也会计算另一组系数。

D 用两份外积理解共享权重

```
contribution_1 = np.outer(X[0], g['A'][0])
contribution_2 = np.outer(X[1], g['A'][1])
np.testing.assert_array_equal(contribution_1 + contribution_2, g['W1'])
np.testing.assert_array_equal(g['W1'], [[-7.375, -50.75], [1.375, 59.75]])
np.testing.assert_array_equal(g['b1'], [[3.375, 56.75]])
wrong_bias = g['A'].mean(axis=0, keepdims=True)
np.testing.assert_array_equal(wrong_bias * 2, g['b1'])
```

检查点D：解释第一份外积为何是 $[-2, 3]$ 、 $[-4, 6]$ ，第二份为何是 $[-5.375, -53.75]$ 、 $[5.375, 53.75]$ 。偏置梯度沿样本轴求和；在已经包含损失平均系数的G_A上再取mean，会得到正确梯度的一半。

E 对九个参数和四个输入逐项差分

```
from experiment import central_gradients
numeric = central_gradients(X, params, Y, 1e-5)
for name in ('W1', 'b1', 'W2', 'b2', 'X'):
    assert numeric[name].shape == g[name].shape
    np.testing.assert_allclose(numeric[name], g[name], rtol=0, atol=2e-8)
```

检查点E：查difference_scan.csv，指出大步长时误差、中等步长时误差及无法分辨时的状态。最大误差取自13个独立坐标，不是只检查一个好看的元素。1e-17是故意的失败输入，会记录Unresolvable input step，并留下空误差字段，不应写作零。

数值差分每次复制当前数组，仅改一个独立元素，然后重新算全部损失。接口还检查h为正有限实数、2h可表示、扰动前后可区分及中间结果有限；这不是支持全部float64范围的稳定性保证，极小量下溢也可能影响精度。

F 独立建立小雅可比并核对两种乘积

```
from experiment import parameter_jacobian, flatten, jvp, vjp
J = parameter_jacobian(X, params)
assert J.shape == (2, 9)
np.testing.assert_allclose(g['P'].ravel() @ J, flatten(g), rtol=0, atol=1e-12)
directions = {k: np.ones_like(value) * 0.1 for k, value in params.items()}
output_direction = jvp(X, params, directions)
np.testing.assert_allclose(output_direction.ravel(), J @ flatten(directions),
atol=1e-12)
weights = np.array([[1.0], [2.0]])
pulled = vjp(X, params, weights)
left = float(np.sum(weights * output_direction))
right = float(flatten(pulled) @ flatten(directions))
assert abs(left - right) < 1e-12
assert abs(left + 0.775) < 1e-12
```

检查点F: JVP的方向只作用于四组参数, X固定。VJP的weights是输出权重, 可以与损失产生的G_P不同。解释两者为什么都不需要由使用者先传入完整J。parameter_jacobian仅用于小教学检查, 超过100000个元素主动拒绝; 本实验不把小例子的建表方法推荐给大模型。

G 改变批次时区分共享与独立变量

```
X_twice = np.concatenate([X, X], axis=0)
Y_twice = np.concatenate([Y, Y], axis=0)
g_twice = gradients(X_twice, params, Y_twice)
assert loss(X_twice, params, Y_twice) == loss(X, params, Y)
for name in params:
    np.testing.assert_array_equal(g_twice[name], g[name])
np.testing.assert_array_equal(g_twice['X'], np.concatenate([g['X']/2, g['X']/2]))
```

检查点G: 复制数据后, 损失求和项翻倍、N也翻倍, 所以损失与共享参数梯度不变; 每个输入位置仍是独立变量, 各自只得到原输入梯度的一半。将所有梯度一概宣称不变, 忽略了变量是否共享。

H 三个故意错误与拒绝输入

在一个新单元格里独立构造错误结果，保留正确代码。错误一是对G_A取mean；错误二是把G_A误写成G_H；错误三是把 $X.T @ G_A$ 换成 $X.T * G_A$ 。这三种错误结果都可能与各自正确结果具有相同shape，但数值不同：偏置为(1,2)，另两项为(2,2)。用纸笔值和有限差分分别指出错误，而不要让错误实现成为自己的基准。

```
wrong_A = g['H']
wrong_W1 = X.T * g['A']
assert wrong_W1.shape == g['W1'].shape
assert np.max(np.abs(wrong_W1 - g['W1'])) == 47.75
assert np.max(np.abs(wrong_A - g['A'])) == 43.0
assert np.max(np.abs(wrong_bias - g['b1'])) == 28.375
```

检查点H：以上三个最大误差来自三种独立错误，不是同一次损失的相加分解。把错误W1值与13坐标差分中的W1部分比较，指出差分结果接近哪一组。解释为什么“shape相同”只能排除一部分错误。

尝试将标签Y写成(2,)的一维数组、把b1写成(2,1)、在输入中混入True、传入NaN、将h设为0。它们应明确拒绝。已有数值ndarray若早已把布尔值提升成数字，接口无法恢复丢失的历史；不要把当前dtype检查误说成类型来源审计。

I 提交解释并记录范围

提交手算、各shape和计算顺序，三种错例的定位过程，差分步长表，显式J与两种乘积的一致性，以及复制批次后的解释。Notebook与脚本的三份输出应逐字节一致，环境文件记录实际运行版本。

本单元的Notebook已在新Python进程中的真实IPython InProcessKernel按顺序执行；构建环境限制socket，因此未测试浏览器Jupyter界面和跨进程内核传输，也没有实际新装Anaconda或跨平台环境。environment.yml提供学习者环境说明，不把安装说明等同于安装验证。图6只按公式计算数组字节数，没有分配巨型雅可比，也不报告峰值内存或训练速度。